上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/11/03

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

应用程序生命周期管理

模块 7 · 从运维的角度解决问题（2/2）

饮 水 思 源 • 爱 国 荣 校

SJTU

从运维的角度解决问题 – 生命周期管理

本讲位置：从「坏了怎么办」到「怎样保持、怎样安全地改」

上一讲 · L-12

硬件故障靠冗余吸收；约 70% 的故障由线上变更引起，常常同时落在所有副本上

L-12 第 15 页的对策：渐进放量、快速发现、安全回退。本讲展开

本讲问题

运行中的应用怎样一直保持在期望状态，又怎样安全地改变它?

引例 模块 6 结束时的平台（设定）

A 自愈掩盖了问题 v1.6 三周前发布，新增的定时任务每分钟刷新一次课程缓存，旧缓存的堆外副本没有释放（每次约 2.6 MB）。每个 Tomcat 约 6 小时涨到 5.5 GB 的 limit，被 OOMKilled 后自动重启；三周共重启约 170 次，外部探活的可用性 100%。每次重启记一张工单（L-08 第 31 页），170 张都以「有冗余，不影响」关闭。**两周后是开放日。**

B 变更被放大 上周把存放 session 的 Redis 迁到新集群，配置中心 30 秒内把新地址推到全部 Tomcat。新集群的安全组没有放通，所有实例的就绪检查都失败，**全站 502 约 6 分钟**，用掉本月为发布与演练预留的 20 分钟的 30%。

本讲结构

1 期望状态
2 健康检查与自愈
3 变更的执行
4 故障处置
5 课程实践

课程实践第 1、2 题分别回答引例 A、B

本模块 从运维的角度解决问题（L-12 ~ L-13）：硬件与软件（L-12）· 应用程序生命周期管理（本讲）

来源：L-08 第 28、31 页；L-11 第 24 页；L-12 第 15 页；引例中的数字均为设定值或推算（l13_numbers.py）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、期望状态：从脚本到声明

第十三讲 · 应用程序生命周期管理

饮 水 思 源 • 爱 国 荣 校

SJTU

命令式与声明式

项目	命令式脚本	声明式描述
写什么	步骤：「装 JDK，拷 war 包，改配置，重启」	**结果：「2 个 Tomcat，v1.6.1，每个 2.6 核、5.5 GB」**
重复执行	不一定安全：第二次「新建」会报错，或者建出第二份	幂等：已经是期望状态，就什么都不做
有人手工改过	脚本不知道，环境慢慢漂移	工具比较期望与实际，列出差异
谁来持续执行	人，执行一次	**控制环持续对齐（L-10 第 21 页）**
回退	再写一个反向脚本	改回上一版声明

重建而非修补（L-05 第 15 页） 环境由声明重建，不在原机上修补；漂移没有藏身之处，staging 与 prod 由同一份声明生成

回顾 L-10 的调度与补建、L-11 的伸缩，都是「把实际拉回期望」的控制环；本讲把同样的方法用到整个应用

要点 声明写的是「要什么」，控制环负责「怎么到」；人改声明，不直接改机器

来源：L-05 第 15 页；L-10 第 21 页

三层声明与 GitOps

层	管什么	代表工具	变更多快
基础设施	云主机、网络、云盘、负载均衡	Terraform、OpenTofu（开源分支）；华为云资源编排 RFS（支持 Terraform 的 HCL 模板，用执行计划预览差异）	分钟到小时
主机配置	软件包、内核参数、用户	Ansible（模块按幂等设计）	分钟
应用	版本、副本数、配额、探针、放置	Kubernetes 清单	秒到分钟

GitOps（OpenGitOps 原则 v1.0.0）

期望状态以声明表达；存放在有版本、不可篡改的地方，保留完整历史；由代理自动拉取；持续观察实际状态，向期望状态对齐

变更即一次合并：评审、门禁（实验 1 的合并门禁）、可追溯

状态与锁

Terraform 用状态文件记录「声明 ↔ 实际资源」的对应

可能写状态的操作都会加锁，拿不到锁就不继续：防止两个人同时修改同一份环境

要点 三层各有工具，原则相同：声明进仓库，评审后由工具执行，工具持续对齐

来源：OpenGitOps, GitOps Principles v1.0.0；Terraform docs: State Locking；华为云 RFS 产品介绍（2026-10-27 查阅）；实验 1

一段声明，与它的代价

L-10 第 27 页的放置方案写成声明（Kubernetes 清单，省略标签与选择器）

```
apiVersion: apps/v1
kind: Deployment                                  # 副本数交给伸缩器（L-11）
metadata: {name: tomcat}
spec:
  strategy:                                       # 发布方式（第三节）
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    spec:
      topologySpreadConstraints:                  # 放置（L-10）
      - {maxSkew: 1, topologyKey: kubernetes.io/hostname,
         whenUnsatisfiable: DoNotSchedule}
      containers:
      - name: tomcat
        image: registry.example/enroll:v1.6.1     # 版本（L-05）
        resources:                                # 配额（L-09）
          requests: {cpu: "2.6", memory: 5.5Gi}
          limits:   {cpu: "4",   memory: 5.5Gi}
        readinessProbe: {httpGet: {path: /ready, port: 8080}}
```

每一行都对应前面的一张表

版本：制品（L-05）
配额：配额表（L-09）
放置：放置方案（L-10）
伸缩：弹性方案（L-11）
发布方式：本讲第三节

代价

要学工具，要管状态文件与锁

声明与控制器要分工：启用 HPA 后，清单里不写 spec.replicas，否则每次同步都会把伸缩器调好的副本数改回去

一处写错，处处生效：症状 B 也是一次声明式的修改，一行配置 30 秒推到全部实例

要点 声明式管理把「改机器」变成「改一行文字」，变更管理因此更重要，而不是更不重要

来源：Kubernetes docs (v1.37)：Deployments、Pod Topology Spread Constraints、Horizontal Pod Autoscaling（2026-10-27 查阅）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、健康检查与自愈

第十三讲 · 应用程序生命周期管理

饮 水 思 源 • 爱 国 荣 校

SJTU

自愈的四层

层	由谁执行	怎么发现	多久（本平台）	管不了什么
进程	systemd：Restart=on-failure	进程退出	默认等 100 ms 再启动，加启动时间	进程还在，但已卡住
容器	kubelet，按重启策略	进程退出；存活检查失败	重启加预热约 1 min；反复失败时退避	节点坏了
实例	控制器补建（Deployment）	实例消失；节点失联	约 7 min：50 s 判定、300 s 后驱逐、再启动；要有空位（L-10 第 16 页）	所有实例一起坏
节点	云平台：规避宿主机故障，或生成重启、重部署事件（L-12 第 6 页）	宿主机与节点的健康检查	分钟级；加节点约 4 min（L-11）	软件缺陷

层次越低越快、越局部；层次越高越慢、覆盖越广

哪一层都修不了代码：软件缺陷换个地方，照样出现

要点 自愈把 MTTR 从「等人」缩短到「等机器」，前提是故障局部、相互独立

来源：systemd.service(5)；Kubernetes docs (v1.37)：Pod Lifecycle（2026-10-27 查阅）；L-10 第 16 页；L-11；L-12 第 6 页

探测的算术

检出时间 ≈ 周期 × 失败阈值，再加一次超时

Kubernetes 默认：周期 10 s、失败阈值 3、超时 1 s、成功阈值 1 → 约 30 s；思考题 1 的 Nginx：1 s × 3 次 → 约 3 s

调快则误判多：思考题 1 第四幕，参数调得过敏，一次抖动导致全站 502 三分钟

探针	回答什么	失败后	注意
启动探针 startupProbe	启动完成了吗	继续等；超过上限则重启	通过之前不做另外两种检查；Tomcat 启动 10 s ~ 2 min（L-05 第 22 页），周期 10 s 时失败阈值不少于 12
存活探针 livenessProbe	进程还能自己恢复吗	重启容器	只查本进程；Kubernetes 文档提醒：实现不当会在高负载下引发连锁重启
就绪探针 readinessProbe	现在能接流量吗	从负载均衡摘除，不重启	**依赖不可用只影响就绪（L-05 第 23 页）；但共享依赖一坏，所有实例同时未就绪（症状 B）**

要点 三种探针各问一个问题；问错了，就会重启不该重启的、摘除不该摘除的

来源：Kubernetes docs (v1.37)：Liveness, Readiness, and Startup Probes（2026-10-27 查阅）；L-05 第 22、23 页；思考题 1

自愈的边界：重启会掩盖慢性问题

症状 A 的账

平时 RSS 约 4.6 GB，距 limit 余 0.9 GB；泄漏约 156 MB/h → 约 5.9 h 被杀一次；三周两个实例约 171 次

每次被杀，至多约 50 个请求失败（摘除前发给它的与在途的）；三周合计约 0.016%，低于提交成功率 SLO 允许的 0.1%

探活总有一个实例应答，可用性 100%

两个实例被杀的时刻相隔 80 s（v1.6 发布时的启动间隔；泄漏与流量无关，间隔不变），恢复要 60 s：**100% 只是因为恰好错开，余量只有 20 s**

按 l13_numbers.py 推算

掩盖了什么

泄漏一直在；症状层的指标（探活、成功率）被冗余吸收

原因层的信号（重启次数、OOMKilled、内存曲线）都有，但被逐条处理

问题不在单次重启，而在重启的频率和趋势

自愈要会停手

Kubernetes：连续崩溃时，重启前依次等 10、20、40 s……最长 300 s（CrashLoopBackOff），连续运行 10 min 后重置

systemd：默认 10 s 内启动超过 5 次即放弃，交给人处理；v254 起可配置指数退避（RestartSteps=）

PodDisruptionBudget 只管主动驱逐，管不了一起 OOM（L-10 第 16 页）

要点 自愈解决「坏一个」，掩盖「一直在坏」：按频率和趋势告警，重启无效时停手

来源：Kubernetes docs (v1.37)：Pod Lifecycle；systemd.service(5)、systemd-system.conf(5)（2026-10-27 查阅）；L-10 第 16 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

三、变更的执行

第十三讲 · 应用程序生命周期管理

饮 水 思 源 • 爱 国 荣 校

SJTU

从运维的角度解决问题 – 生命周期管理

变更 • 控制环

每次变更都是一个小控制环



分批的两个参数

maxSurge（最多多开几个）、maxUnavailable（最多少几个），默认都是 25%：前者向上取整，后者向下取整

2 个副本：1 和 0；6 个副本：2 和 1

发布用时（L-05 第 24 页的公式）：平时 2 个 Tomcat 逐个替换，每个「拉镜像 20 s + 启动预热 60 s + 排空 ≤ 30 s」，共约 3.7 min

谁来决定回退

Deployment 不会自动回退：progressDeadlineSeconds（默认 600 s）只把发布标记为失败

「观察 → 回退」要由门禁完成（下一页）

发布、配置修改、节点维护、缩容都是这个环，差别在一批多大、看什么、看多久

要点 少了「观察」和「回退」，分批也只是慢一点地全部坏掉

来源：Kubernetes docs (v1.37)：Deployments（2026-10-27 查阅）；L-05 第 24 页；实验 1；用时按 l13_numbers.py 推算

放量门禁：只在一小批里出问题

金丝雀与判据

先 1 个实例，再全部（L-05 第 16 页）

判据用 SLI（L-08 第 28 页：提交成功率、提交 p99），按版本拆开比较

越线即自动回退，不等人

观察多久

观察时间要覆盖故障出现所需的时间

配置错误：几秒就出现

内存泄漏：要几小时。v1.6 的 RSS 每小时涨约 156 MB，观察 2 h 相差约 312 MB

重放症状 B：配置也要分批

先推 1 个实例 → 它的就绪检查失败，约 3 s 被摘除 → 自动回退

另一个实例照常服务：至多约 50 个请求失败，预算约为 0

配置与代码一样分批（L-05 第 15 页）；功能开关也是配置（L-05 第 16 页）

代价

发布变慢

要有按版本拆开的指标

判据太松拦不住，太紧总在回退

要点 门禁把「全部出问题以后再发现」变成「只在一小批里出问题，而且自动退回」

来源：L-05 第 15、16 页；L-08 第 28 页；数字按 l13_numbers.py 推算

计划内的中断：先算剩下的够不够

主动与非主动中断

主动：排空节点、平台的计划事件（L-12 第 6 页）、缩容（L-11 第 15 页）

非主动：崩溃、OOM、节点故障

排空（drain）

先把节点标为不可调度，再逐个驱逐实例；每次驱逐都是一次优雅停止（L-05 第 24 页）

被驱逐的实例要有地方放：放置规则在维护时仍然生效（L-10 第 16 页）

PodDisruptionBudget

限制主动驱逐时同时缺席的实例数，kubectl drain 遵守它

不限制 OOM、节点故障，也不限制滚动更新（由 maxUnavailable 控制）

按容量定，不按副本数定：minAvailable 等于副本数时允许中断 0 个，排空永远完不成

有状态组件

数据在哪儿，实例就得在哪儿（L-10 第 13 页）

L-11 第 18 页留下的问题：MySQL 的规格或磁盘怎么调整？平台原地变更、停机迁移、主从切换

比较写中断、准备时间、回退方式和由谁执行（L-12 第 6 页的接口）；数字见课程实践第 4 题

要点 先算剩下的实例够不够、维护期间还有没有冗余，再决定一次停多少

来源：Kubernetes docs (v1.37)：Disruptions、Specifying a Disruption Budget、Safely Drain a Node；L-05、L-10～L-12

错误预算决定节奏

本月为发布与演练预留 20 min（L-08 第 28 页）

症状 B：6 min

剩 14 min

冻结

预算耗尽即冻结（L-08 第 21 页）

开放日前一周不变更（L-08 第 28 页）

冻结之前要留出缓冲，用于回退和复测

窗口与口径

按时间计的 SLI（外部探活）：凌晨停 4 分钟也算 4 分钟

按请求计的 SLI（成功率）：凌晨几乎不花预算（L-08 第 19 页）

SLI 的口径决定了维护窗口的价值

一次一件，还是合并

分开做：便于归因和回退

合并做：少一次窗口、少一次中断

更好的问题：能不能把中断变成不中断（在线变更、主从切换）？

要点 错误预算把「能不能改、什么时候改、一次改多少」变成算术

来源：L-08 第 19、21、28 页；数字按 l13_numbers.py 推算

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、故障处置

第十三讲 · 应用程序生命周期管理

饮 水 思 源 • 爱 国 荣 校

SJTU

时间线：先止血，再找根因

症状 B 的时间线（分:秒，设定）



0:00 推送新地址

0:30 全部未就绪，探活连续失败，通知值班
全站 502

1:00 探活连续失败，通知值班

5:30 定位到配置修改，回退

6:30 恢复

MTTR 按段拆开：发现 0.5 min + 定位 4.5 min + 修复 1 min = 6 min；**定位占 75%**

缩短每一段

发现：按症状告警（L-08 第 31 页）

定位：变更记录与告警放在一处，先问「最近改了什么」（L-12 第 15 页）

修复：回退一步完成，事先演练过

先止血，再找根因

回退最近的变更、切走流量、降级（只读、关闭非核心功能）、限流（L-09 第 22 页）；同时保留证据：日志、配置版本、时间线

Google SRE Book 第 14 章：「Stop the bleeding, restore service, and preserve the evidence for root-causing.」

要点 MTTR 里最长的通常是定位，而定位最依赖事前的准备

来源：Google SRE Book, Ch. 14 Managing Incidents（2026-10-27 查阅）；L-08 第 31 页；时间线按 l13_numbers.py 推算

角色与复盘

什么时候宣布事故

需要第二个团队参与?

用户能看见?

集中分析一小时仍未解决?

任何一项为「是」即宣布

四个角色

指挥：掌握全局、分派工作

操作：事故期间唯一修改系统的人

沟通：向教务处、学生通报

计划：记录、处理长期事项

小团队可以一人兼几个角色，但修改系统的只能有一方

复盘

触发：用户可见的中断或降级超过阈值、任何数据丢失、值班人工干预（回退、切流量）、恢复时间超过阈值、监控没有发现问题

不追责：假设每个人都出于好意，按当时掌握的信息行事

产出：影响、处置过程、根因，和带负责人与期限的行动项

DORA 失败部署恢复时间（L-05 第 7 页：精英团队小于一小时）衡量的正是这一节

要点 事故处置是事先排练过的流程；复盘的产出是行动项，不是责任人

来源：Google SRE Book, Ch. 14 Managing Incidents、Ch. 15 Postmortem Culture（2026-10-27 查阅）；L-05 第 7 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：一周的变更计划

第十三讲 · 应用程序生命周期管理

饮 水 思 源 • 爱 国 荣 校

SJTU

参数设定

沿用

平台：平时 2 个节点、2 个 Tomcat，开放日 3 个节点、6 个（L-11 第 24 页）；拓扑分布 maxSkew 1、DoNotSchedule（L-10 第 27 页）

Tomcat：request 2.6 核、limit 4 核，内存 5.5 GB，开放日峰值 RSS 5.1 GB（L-09 第 14、29 页）；拉镜像约 20 s，启动预热约 1 min，优雅停止上限 30 s

MySQL：单实例、专用节点，通用型SSD，fsync 约 2 ms，写上限约 250 TPS；L-12 的结论是换极速型SSD（fsync 约 0.5 ms，写上限 1000 TPS，容量线 700）；写入口限流 250 TPS（L-11 第 24 页）

SLO：可用性 99.9%（外部探活每 10 s 一次），提交成功率 ≥ 99.9%；平时每月为发布与演练预留 20 min；开放日前一周冻结（L-08 第 28 页）

新增（本讲设定）

平时约 30 QPS；症状 A、B 见第 3 页

下一个开放日在两周后，冻结之前还有 7 天（D1～D7）；本月预留的 20 min 已用 6 min

四项待办

1 v1.6.1：修复泄漏

2 内核安全补丁：两个应用节点和 MySQL 节点都要重启；云主机重启约 3 min，MySQL 停止、启动各约 30 s

3 MySQL 云盘由通用型SSD 换成极速型SSD，数据约 40 GB。三种做法：平台在线变更（华为云「变更云硬盘类型」，公测，需工单申请）；停机拷贝（约 150 MB/s）；建从库后主从切换（写入中断约 30 s，建从库需数小时）

4 写入口限流由 250 TPS 调到 700 TPS

注 数值为设定，与 L-07 的资源账、L-09～L-12 的配额、放置、弹性方案和写路径一致

来源：L-08 第 28 页；L-09 第 14、29 页；L-10 第 27 页；L-11 第 24 页；L-12 第 23～28 页；华为云 EVS 文档

课堂讨论

1 自愈

症状 A 中可用性 100%，170 张重启工单逐张关闭，问题出在哪一层的指标和告警上？重启应当怎样进入告警？

开放日的负载下会发生什么？自愈什么时候应当停手、交给人？

2 变更放大

症状 B 的 6 分钟里，哪一步本可以拦住它？为这次配置修改写出分批步骤和自动回退的判据。

所有实例同时就绪失败时，负载均衡应该全部摘除吗？

3 节点补丁

逐个排空并重启两个应用节点：PodDisruptionBudget 设为 minAvailable 2 会怎样？被驱逐的 Tomcat 去了哪里？大约要多久？

MySQL 节点怎么办？如果必须在开放日那周打补丁，6 个 Tomcat 时怎么设？

4 一周的计划

换盘的三种做法各中断写入多久、用掉多少预算？

四项变更按什么顺序、在哪一天做？各自的判据和回退预案是什么？做完还剩多少预算？

安排 每题小组讨论 2 分钟、汇报 1 分钟，其余时间讲评

从运维的角度解决问题 – 生命周期管理

讨论要点（1） （2）

1 自愈：看频率和趋势

哪一层 症状层的 SLO（探活、成功率）被冗余吸收，看不见；原因层的信号（重启次数、OOMKilled、内存曲线）都有，但逐张关闭。单次重启确实不影响，影响的是频率和趋势

告警 按频率与原因：同一版本一天内 OOMKilled 超过若干次，或按内存斜率预测「几小时后到 limit」，记工单并指定负责人；多个实例同时重启或燃烧率上升时通知值班

kube-state-metrics：重启次数 kube_pod_container_status_restarts_total；上次终止的原因 kube_pod_container_status_last_terminated_reason（实验性）

开放日 峰值 RSS 5.1 GB，余量 0.4 GB，间隔由 5.9 h 缩到 2.6 h。2 个老实例相位只差 80 s，约 56% 的可能在 9:00 前后一起被杀（8:50 新建的 4 个不会）。剩 4 个共 11.2 核 ≥ 10.2 核，扛得住；但这 1 分钟里再坏一个，只剩 8.4 核 < 8.7 核：**冗余被泄漏用掉了**

对策与停手 冻结前上线 v1.6.1；来不及时，开放日前（如 8:40）滚动重启 2 个老实例（止血，问题仍在）。同一原因反复出现、刚启动就失败，说明重启无效，应退避并交给人

2 变更放大：分批、判据、不全摘

拦截点 切换之前，从一个实例检查新地址的连通性；配置分批，先推 1 个实例；告警页直接列出「最近的变更」，缩短 4.5 min 的定位

分批与判据 1 个实例 → 观察 5 min → 全部。判据：该实例的就绪状态、按实例拆开的 5xx 与 Redis 连接错误（L-08 第 6 页）、提交成功率；越线自动回退。这样做，这台约 3 s 被摘除，另一台接住，预算约为 0

全部摘除吗 共享依赖进入就绪检查，依赖一坏所有实例同时未就绪，摘除只是把「部分可用」变成「全部 502」（又一次共因故障，L-12 第 14 页）

对策 大部分实例不健康时不再摘除：Envoy 的 panic threshold，健康比例低于 50%（默认）时忽略健康状态，向全部实例分流；就绪检查只查本实例能否服务，共享依赖故障时降级，不依赖 session 的课程查询照常服务

预算：6 min 是平时预留的 30%、每月预算的 14%

结论 自愈要看频率和趋势；变更要分批、有判据、能自动回退，共享依赖不进就绪检查

来源：按 l13_numbers.py 推算；kube-state-metrics docs: Pod Metrics；Envoy docs: Panic threshold；L-08 第 6 页；L-12 第 14 页

讨论要点（3）

问题	要点
minAvailable 2	等于副本数，允许中断 0 个，排空永远完不成；平时取 minAvailable 1
被驱逐的 Tomcat 去了哪里	另一个节点按内存还能放 1 个，但拓扑分布把被排空的节点也计入（nodeTaintsPolicy 默认为 Ignore）：放过去两边相差 2，违反 maxSkew 1，于是 Pending，直到节点重启回来。这段时间只有 1 个 Tomcat：容量够（30 QPS 约 0.09 核），但没有冗余；要保住冗余，先临时加一个节点（约 4 min）
大约多久	每个节点：排空 ≤ 30 s + 重启 3 min + Tomcat 启动约 1 min ≈ 4.5 min，两个约 9 min；低峰逐个进行，第一个节点回来、Tomcat 就绪后再做第二个
MySQL 节点	单实例：重启即停约 4 min，读写全部中断，按探活计用掉 4 min 预算。可选：凌晨做；内核热补丁（只能替换函数，只适用于部分补丁）；推到开放日之后（取决于漏洞能否被利用、MySQL 是否暴露在外）
开放日那周（6 个）	容量需要 4 个：minAvailable 4，一次排空一个节点；被驱逐的 2 个放不下（每个节点按内存只放 2 个），节点回来之前没有冗余；不在 8:50 ~ 9:30 做

结论 PodDisruptionBudget 按容量定；维护时放置规则照样生效，先想清楚被驱逐的实例去哪里

来源：按 l13_numbers.py 推算；Kubernetes docs (v1.37)：Disruptions、Pod Topology Spread Constraints；内核文档 Livepatch

讨论要点（4）

做法	写中断	准备	预算	回退	风险
平台在线变更	0；性能可能短暂下降	工单申请公测，约 1 天（L-12）	**0**	再变更回通用型SSD，同样在线，用时不定	用时从几秒到数小时，过程不能中断；期间不能快照、备份、扩容
停机拷贝	约 5.6 min	少	5.6 min：剩余 14 min 的 40%	挂回旧盘	中断长
主从切换	约 30 s	建从库需数小时，人日最多	0.5 min	反向复制后切回	应用要改数据库地址：又一次配置变更（症状 B）

选在线变更 先做快照（只用于灾难恢复：回到快照会丢掉之后写入的数据）；低峰进行；看 fsync 延迟与提交 p99。云盘的性能还受云服务器规格的存储 I/O 上限约束

依赖 限流 250 → 700 TPS 必须排在换盘验证之后：盘没换就放开，700 TPS 进来，盘只能提交 250，多出的 450 TPS 排队等锁（L-11 第 23 页）

预算 计划内只有 MySQL 节点补丁用掉 4 min，剩 10 min 留给演练与意外

结论 能把中断变成不中断，就不花预算；有依赖的变更按依赖排序

来源：按 l13_numbers.py 推算；华为云 EVS 文档：变更云硬盘类型（公测）、云硬盘类型及性能介绍；L-11 第 23 页

参考变更计划

日	变更	做法与判据	回退	预算
D1	v1.6.1	金丝雀 1 个实例，看内存斜率 2 h，持平再全量（约 3.7 min）；同日提交换盘的公测工单	滚动回到 v1.6	0
D2	两个应用节点打补丁	低峰，逐个排空，minAvailable 1；每个约 4.5 min	从旧内核启动	0
D3	MySQL 云盘在线变更	先快照；低峰；看 fsync 与提交 p99	变更回通用型SSD	0
D4	写入口限流 250 → 700 TPS	换盘验证之后；分批生效，看提交 p99 与成功率	改回 250	0
D5 凌晨	MySQL 节点打补丁	提前通知，停机约 4 min	从旧内核启动	**4 min**
D6	开放日演练	6 个实例、开放日负载：内存曲线、fsync、限流、扩容时间	—	预留
D7	缓冲	处理回退与复测	—	—

排序的理由 一天一件，便于归因；风险最大、观察最久的先做（v1.6.1）；按依赖排序（换盘在前，限流在后）；花预算的放在验证之后；演练做一次整体验证；冻结前留缓冲

要点 变更计划 = 按依赖与风险排的顺序 + 每项的判据与回退 + 一本预算账

来源：按 l13_numbers.py 推算；本讲第 13 ~ 16 页

从运维的角度解决问题 – 生命周期管理

本讲小结

- **期望状态：声明结果，不写步骤；控制环持续对齐**
  - 代价是状态、锁和更大的影响范围
- **健康检查与自愈：四层各管一段；检出时间约为周期 × 阈值**
  - 自愈会掩盖慢性问题：按频率和趋势告警，重启无效时停手
- **变更的执行：分批、门禁、自动回退**
  - 计划内中断先算剩下的够不够；错误预算决定节奏
- **故障处置：先止血，再找根因**
  - MTTR 按段拆开，定位最长；复盘产出行动项

判断标准 期望状态写在哪儿、由谁持续对齐？这次变更一次影响多少、靠什么发现、多久能退回？

模块 7 总结：从运维的角度解决问题

L-12 硬件与软件

分清问题落在哪一边、由谁解决

判断标准

问题落在硬件还是软件、由谁解决？按月付还是一次付、上限在哪？剩下的故障是独立的还是相关的？

L-13 应用程序生命周期管理

保持期望状态，安全地改变

判断标准

期望状态写在哪儿、由谁持续对齐？这次变更一次影响多少、靠什么发现、多久能退回？

模块 7 回答「运行中的应用怎么管」

先分清问题落在哪一边、由谁负责，再让应用保持在期望状态，并安全地改变它（L-08 第 33 页的预告）

要点 独立的故障靠冗余吸收，相关的故障靠管理变更来控制

来源：L-12 第 29 页；L-08 第 33 页；L-11 第 26 页（模块 6 总结）

全课回顾：七个模块的判断标准

模块	讲义	判断标准
1 互联网产品的特点	L-01	用户规模、流量变化和迭代速度，对系统提出了什么要求？
2 软件架构	L-02	这个架构满足了哪些非功能性需求，牺牲了什么？
3 前后端的演进	L-03	每个模式解决了什么问题，又引入了什么复杂度？
4 复杂应用场景	L-04	当前的瓶颈在哪里？最小的有效改动是什么？
5 重新认识软件	L-05 ~ L-08	产出物是什么、占用哪些资源、上限由什么决定、用什么指标衡量？
6 资源管理	L-09 ~ L-11	给多少、放哪儿、何时变？
7 从运维的角度解决问题	L-12 ~ L-13	落在硬件还是软件？怎样保持期望状态、安全地改变？

方法 瓶颈 → 方案 → 新问题；问题 → 原因 → 对策

贯穿全课 这项技术解决了什么问题，又增加了什么成本？

来源：L-01 第 5 ~ 9 页；L-03、L-04 小结页；L-08 第 33 页；L-11 第 26 页

课后思考与参考资料

课后思考

1. 小组项目的环境是怎么搭起来的？哪些步骤还靠手工？把其中一项写成声明（配置文件或脚本），在一台空主机上重建一次

2. 为小组项目写出存活检查、就绪检查各查什么、多久判定一次。哪些依赖不应该进入就绪检查？

3. 小组项目最近一次出问题：按发生、发现、定位、修复拆开时间线，哪一段最长？写一条复盘的行动项

参考资料

Google SRE Book：Ch. 1 Introduction；Ch. 8 Release Engineering；Ch. 14 Managing Incidents；Ch. 15 Postmortem Culture: Learning from Failure

Kubernetes docs (v1.37)：Liveness, Readiness, and Startup Probes；Pod Lifecycle；Deployments；Specifying a Disruption Budget for your Application；Safely Drain a Node；Pod Topology Spread Constraints

systemd.service(5)；systemd-system.conf(5)

OpenGitOps：GitOps Principles v1.0.0；Terraform docs：State Locking；华为云文档：资源编排服务 RFS

华为云文档：变更云硬盘类型（公测）

kube-state-metrics：Pod Metrics；Envoy docs：Panic threshold；Linux 内核文档：Livepatch

感谢聆听

饮 水 思 源　爱 国 荣 校